

Network Compliance Auditor

Author: Wael Shahadeh

Version: 1.0.0

About This Lab

This lab introduces network engineers to Python by having them build a working network compliance auditor. There is `reference/audit.py` as a guide to what a functional Python program looks like, and `starter/audit.py`, which provides a skeleton script for the engineer learning Python to build. Work in `starter/audit.py` and only use `reference/audit.py` if you get stuck. This guide lists instructions on how to get started with Python and basic usage, teaching how to build a functional `audit.py`.

The architecture diagram can be found in `../report/architecture.png`

Module 0: Setup and Prerequisites

- You would need an Integrated Development Environment (IDE) of your choosing. An IDE is a piece of software that allows you to write, test, and debug code. I recommend Visual Studio Code, which you can download [here](#).
 - There are several extensions which make coding in Python within VSCode for this lab more friendly. Press Ctrl (or Command in macOS) + shift + x to open the extension menu.
 - Gitlens
 - indent-rainbow
 - Python
 - Ruff
 - Yaml
- The software Git is a very useful tool for code management. You can install it [here](#)
 - Once Git is installed, restart Visual Studio Code.
 - On the welcome page of Visual Studio Code, there will be the option to clone a repository. Click that, sign into GitHub (or make an account), and paste the URL of this repository into the search bar and press enter to clone (making a copy) of this repository for you to own locally on your device. Save into whichever folder of choice.

- This program and lab uses Python 3.10 or later, which you can download [here](#).
 - Ensure that when you are going through the installer, you select the "Add to Path" option.
 - Once Python is installed, restart Visual Studio Code.
 - On the top panel of Visual Studio Code, there will be a terminal option.
 - Open a terminal and type `pip install -r requirements.txt`
 - This opens requirements.txt and installs any external libraries this program uses. This lab in particular uses pyyaml and rich, which allow us to read and utilize files in a YAML format and have an elegant terminal output.

Congratulations! You are now ready to code in Python and learn from this lab!

Module 1: Loading the Inventory

Objectives:

- Understand YAML basics
- Use `yaml.safe_load`
- Write your first Python function with help
- Write your second Python Function on your own
- Work with dicts and lists

Background

What is YAML?

YAML is a human-readable data serialization standard, with the goal of being easier to read and understand compared to other data structures such as JSON or XML. YAML uses .yaml or .yml file extensions.

In the context of this lab, we are using YAML for the policy list for network devices and the inventory list. The reason we are using an external YAML file instead of hardcoding it into the Python program is that policies and devices change and update. It is relatively easy to break a program, so for the sake of stability, scalability, and maintainability, it would be wise to keep information of that nature in an external file, and just read the data from those files with our Python program.

YAML has three structural rules:

1. **Key-value pairs** use a colon followed by a space:

```
name: router1
```

2. **Lists** are written with a dash for each item

```
interfaces:
  - eth0
  - eth1
```

3. **Nested data** uses indentation. Items under a parent must be indented the same amount. Always use spaces and never use tabs:

```
router1:
  ip: 10.0.0.1
  role: edge
```

Looking at `reference/inventory.yaml` :

```
devices:
  - name: router1
    role: edge
    site: DC
    config_path: reference/configs/router1.txt
  - name: router2
    role: core
    site: branch1
    config_path: reference/configs/router2.txt
  - name: switch1
    role: core
    site: branch1
    config_path: reference/configs/switch1.txt
```

Notice the structure of `inventory.yaml`. A top key is called `devices` , whose value is a list, which is every item starting with `-` . Each device has four fields:

- `name` - Identifier used in reports
- `role` - `core` or `edge` , used later for rule scoping
- `site` - The physical location, also used for rule scoping
- `config_path` - The relative path to the device's config file

Writing your first Python function!

Open `starter/audit.py` . You'll see this in the first function:

```
def load_inventory(path):
    #Module 1: Load device inventory from YAML.
```

```
#Open the file, parse YAML, return the devices list.  
pass
```

`def` means to define. We are "defining" a function called `load_inventory` for us to use in our program. Inside the paranthesis, we wrote `path`. This is just a placeholder variable that we can input into our function and manipulate. `pass` means to do nothing. It is so we can make this empty function without Python failing. It will be replaced by actual code.

Each step will have instructions, a description of the goal, and a framework. Try to complete each step on your own. The solution would be right below if you get stuck.

There is not a one answer solution for coding 99% of the time! If your function doesn't look identical to the solution but is fully operational, that's good!

Step 1: Safely opening the file

Python opens files with the `open()` function. The issue is, leaving files open and not in use can cause a resource leak, leading to performance issues. The solution is simple! Python has a `with` statement, which handles opening and closing automatically:

```
with open(<path>) as <variable>:  
    # code that uses the file, we can use pass as a placeholder  
pass
```

Try it yourself! Inside the `load_inventory` function, write a `with` block that opens the inventory file at `path` and names the opened file as `f`. You don't need to put stuff inside the block yet.

Solution

```
with open(path) as f:  
    pass
```

Step 2: YAML parsing

The `pyyaml` library has a `yaml.safe_load()` function built in. This opens a YAML file and returns its contents as objects that Python can understand, such as dicts, lists, strings, and numbers.

Try it yourself! Inside your `with` block, store the results of `f` into a variable called `data` using `yaml.safe_load()`. You can get rid of the `pass`.

Solution

```
with open(path) as f:
    data = yaml.safe_load(f)
```

Step 3: Return a device list

`data` now holds parsed YAML. Looking at `inventory.yaml`, its structure is `{"devices": [...]}`. The list of devices is under the `devices` key.

Lets introduce the `return` concept in Python. `return` takes the output of a function and moves that data out of the function and into the rest of your code. A variable is a name that points to a specific spot in memory.

Question: Should the function return `data`, which is the whole parsed YAML, or `data["devices"]`? Why?

Answer: The function should output `data["devices"]` because the caller of `load_inventory` only cares about the device list. They don't need to know that there is a `devices` key wrapping it. Returning the list directly hides that detail.

Try it yourself! Add a `return` statement that gives you back just the lists of devices.

```
with open(path) as f:
    data = yaml.safe_load(f)
    return data["devices"]
```

That is `load_inventory` complete!

Now build `load_policy`!

Using what we learned so far, build `load_policy` so that it reads `policy.yaml` safely and outputs the needed data!

Hint: `load_policy` should output the full parsed YAML dict, not just a subkey! The caller needs both metadata and rules.

Testing the function

Add a test block at the bottom 'starter/audit.py':

```
if __name__ == "__main__":
    inventory = load_inventory("starter/inventory.yaml")
    print(f"Loaded {len(inventory)} devices")
```

```
for device in inventory:
    print(f"    - {device['name']} ({device['role']}) at {device['site']}
```

Now run it from the project root or Visual Studio Code terminal:

```
python starter/audit.py
```

You should see:

Loaded 3 devices

- router1 (edge) at DC
- switch1 (core) at branch1
- router2 (core) at branch1

If this appears in the terminal, your function works. If you see an error, check:

- Did you save `audit.py` after editing?
- Are you running from project root, and not inside `starter/` ?
- Did `pip install -r requirements.txt` complete successfully?

Some common mistakes are using tabs instead of spaces in YAML for indentation. If you use tabs, you'll confuse the parsing.

If your code gets a `KeyError` with `devices` , it means your YAML doesn't have a top-level named `devices:` key.

Final Tasks for Module 1:

1. Add a fourth device named `firewall1` to `starter/inventory.yaml` with `role: edge` and `site: DC` . What does your test output show now?
 2. What's the difference between `yaml.safe_load()` and `yaml.load()` ? Why should you always use `safe_load` ?
 3. What does `load_inventory()` return `data["devices"]` instead of just `data` ?
-

Module 2: Loading Device Configs

Objectives:

- Read raw text from files

- Understand when to use raw file reading vs structured parsing
- How to handle missing files
- Pass and use dict data inside a function

Background

Module 1 used `yaml.safe_load()` because YAML is structured data. It has keys, lists, and nested objects that cleanly work with Python dicts and lists. A device configuration is different in a sense that it is just text. There is no formal structure with plain text files. Using `yaml.safe_load()` on a non-YAML formatted file would either fail or produce garbage data.

Python has a tool built in for raw text files. `f.read()` returns the entire file's contents as a string, which is what we want. We can then search through that string to find whichever specific data to check for compliance patterns.

Looking at `reference/config/router1.txt`:

```
!
! Cisco IOS Configuration
! Device: router1
! Role: edge
! Site: DC
!
hostname router1
!
banner login ^
WARNING: Authorized access only. All activity is monitored and logged.
Unauthorized access is prohibited and will be prosecuted.
^
!
ntp server 10.0.0.100
ntp server 10.0.0.101
!
snmp-server community NetMon-r0 RO
snmp-server location DC-Rack-12
snmp-server contact awesomesauce@example.com
!
ip ssh version 2
!
interface Loopback0
    description Management Loopback
    ip address 10.255.0.1 255.255.255.255
!
```

This is plain text. There are no keys or structures to grab from, just what you would see if you run `show running-config` on a real Cisco device. We want to load it into our program with a function so we can search it later.

Writing `load_config()`

Open `starter/audit.py` and find `load_config()` :

```
def load_config(device):  
    #Module 2: Read a devices config file as a string  
    pass
```

Notice how there is a difference to this function than what we saw in `load_inventory` . We are now using a parameter called `device` instead of `path` . This is because `load_config` receives a whole device dict, which was loaded in from `load_inventory` from Module 1, and the path lives inside that dict.

We will also build `load_config` in three steps.

Step 1: Get the path from the device dict

When `load_config` is called later, it will pass a device dict that looks like this:

```
{"name": "router1", "role": "edge", "site": "DC", "config_path": "referer
```

You can reach a value in the dict by writing `dict_name["key"]` . SO if your dict is named `device` and you want the value at `"config_path"` , you would write `device["config_path"]` .

Try it yourself! Inside `load_config` , write a `with` block that opens the file at the device's config path and names the open file `f` . Use the same pattern as what we learned from Module 1.

Solution:

```
load_config(device):  
    with open(device["config_path"]) as f:  
        pass
```

Step 2: Read the file's contents

Python has a built-in function, `f.read()`, which returns the entire file as a single string. We are using that instead of `yaml.safe_load()` because we are reading an unstructured, regular plain text file.

Try it yourself! Inside your `with` block, store the results of `f.read()` in a variable named `data` ..

```
load_config(device):  
    with open(device["config_path"]) as f:  
        data = f.read()
```

Step 3: Returning the contents

We now want to get the config text from `load_config`. Return `data`.

Try it yourself! Add a `return` statement

Solution

```
load_config(device):  
    with open(device["config_path"]) as f:  
        data = f.read()  
    return data
```

The `load_config` is now complete!

Testing `load_config`

Update your text block of `starter/audit.py`:

```
if __name__ == "__main__":  
    inventory = load_inventory("starter/inventory.yaml")  
    first_device = inventory[0]  
    config = load_config(first_device)  
    print(f"Config for {first_device['name']}: {len(config)} characters")  
    print("First 200 chars:")  
    print(config[:200])
```

We are doing two special things with this test:

- `inventory[0]` grabs the first item from the device list. Python counting starts at `0` instead of `1`.

- `config[:200]` is string slicing. It grabs the first 200 characters of the string. This is useful for previewing a file without outputting its entire contents.

Run the project the same way we did for Module 1 and you should see:

```
Config for router1: 961 characters
First 200 chars:
!
! Cisco IOS Configuration
! Device: router1
! Role: edge
! Site: DC
!
hostname router1
!
banner login ^
WARNING: Authorized access only. All activity is monitored and logged.
```

What happens if someone removes `router1.txt` and you run the script? Try it out! Rename `router1.txt` to `router1_old.txt` to simulate it being removed and run the script.

You will see:

```
FileNotFoundError: [Errno 2] No such file or directory: 'starter/configs/
```

The script crashed! In a real audit, one device with a missing config should not kill the entire audit. We would instead want to record the error and keep going.

Python provides `try/except` for these type of situations:

```
try:
    # Code that has the possibility of failing
    something_risky()
except SomeErrorType:
    # Instructions if it fails
    # error_handler()
```

We won't add error handling to `load_config` itself, which would complicate the function. Instead, we will add it to our `run_audit` function later, where the error context would matter more.

Rename `router1_old.txt` back to `router1.txt` and confirm the script works again.

A common mistake is passing the wrong data type. `load_config` expects a dict, not a string path. If you accidentally call `load_config("configs/router1.txt")`, Python will try to look up `"config_path"` inside the string `"configs/router1.txt"`, which will fail.

Another common mistake is using `yaml.safe_load()` on a config file. The config files aren't YAML, so they won't parse properly.

A note on using this program in production

This lab assumes you would have a config file in the correct directory. In a production environment, you would pull the configuration over SSH, using another Python library like Netmiko. This is beyond the scope of this lab, but is important to mention. The rest of the audit engine does not care about where the config file comes from, so long as it gets its data.

Here is a rough idea of what `load_config` would look like in production. Don't write this in the lab, it is just to represent the usage of Netmiko.

```
from netmiko import ConnectHandler

def load_config(device):
    connection = ConnectHandler(
        device_type="cisco_ios",
        host=device["host"],
        username=device["username"],
        password=device["password"],
    )
    config = connection.send_command("show running-config")
    connection.disconnect()
    return config
```

Some final questions for Module 2

1. Why does `load_config` use `f.read()` instead of `yaml.safe_load()`? What would happen if you used `yaml.safe_load()` on a Cisco config file?
 2. The function takes a `device` dict, not a path string. Why is that a better design instead of just passing the path?
 3. If you wanted to read the config line-by-line, what would you use instead of `f.read()`? When might it be useful?
-

Module 3: Searching Configs for Patterns

Objectives:

- Use Python's `in` operator for substring matching
- Understand when substring matching isn't enough
- Use the `re` module for regular expression matching
- Write a reusable helper function

Background

In Module 2 we loaded a device's config as a single string. Now we need to search that string and find out if specific patterns appear in it. This is part of the core of our audit engine.

1. `in` checks if one string is a substring of another. `in` is exact match only.
2. Regular Expressions (regex) is a flexible pattern-matching language. Regex allows you to match patterns like "the word telnet appears anywhere on a line starting with transport input".

How does `in` work?

The `in` operator is built into Python and is used between two strings. After comparing the strings, it gives a boolean output, being `True` or `False`. It outputs `True` if the left of the string is in the right of the string. For example:

```
"hello" in "hello world" #True
"world" in "hello world" #True
"goodbye" in "hello world" #false
"Hello" in "hello world" #false
```

It is case sensitive. This is fine for our goal of Cisco auditing, because Cisco syntax is consistent.

Regular Expression (regex)

Sometimes substring matching isn't flexible enough. Let's say there is a rule such as `telnet must not be enabled on VTY lines`. The config might say:

- `transport input telnet telnet only`
- `transport input ssh telnet` both telnet and ssh
- `transport input telnet ssh` both but different order

These three strings are all violations of that rule. A substring check for `transport input telnet` would catch the first two strings, but miss the third. A regex like `transport input.*telnet`

catches all three because `.*` means "any characters in between"

Some important regex syntax to know:

- `.` matches any single character
- `.*` matches zero or more of any character
- `(option1|option2)` matches either option (alternation)
- `^` matches the start of a line
- `$` matches the end of a line

Python's `re` module provides the regex functions we need. We will use `re.search()`, which scans the string for the first match and returns a match object.

```
import re

re.search("hello", "hello world") #true
re.search("missing", "hello world") #false
re.search("transport input,*telnet", "transport input ssh telnet") #true
```

Writing `search_config()`

Open `starter/audit.py` and find `search_config()`:

```
def search_config(config, pattern, match_type="exact"):
    #Module 3: Search a config for a pattern. Return True if found
    pass
```

Notice how the function has a third parameter, `match_type="exact"`. This means that it will default to exact searches if the user omits that parameter. If the user does need regex, they can select that and the function will still function properly. This ensures the function works for both simple and complex string searches.

We will build `search_config()` in two steps.

Step 1: Handle exact matching

When `match_type` is `"exact"`, we use `in`. This would return `True` if `pattern` is in `config`, and `False` if not.

Try it yourself! Inside `search_config`, write an `if` statement that checks `match_type`. If it's `"exact"`, return whether `pattern` is in `config`.

Solution

```
def search_config(config, pattern, match_type="exact"):
    if match_type == "exact":
        return pattern in config
```

Step 2: Handle regex matching

When `match_type` is `"regex"`, we use `re.search()`. The function returns a match object on hit, `None` on miss. We want a boolean, so compare to `None`:

```
re.search(pattern, config) is not None
```

This ends up being `True` if there is a match, `False` if not.

Make sure `import re` is at the top of your file. Then add an `else` branch for the regex case.

Try it yourself! Add an `else` clause to `search_config`. Inside it, return whether `re.search(pattern, config) is not None`.

Solution

```
def search_config(config, pattern, match_type="exact"):
    if match_type == "exact":
        return pattern in config
    else:
        return re.search(pattern, config) is not None
```

`search_config` is complete!

Testing `search_config`

Update your test block with this and run it:

```
if __name__ == "__main__":
    inventory = load_inventory("starter/inventory.yaml")
    config = load_config(inventory[0]) # router1's config

    # Exact match - should return True
    print("NTP server present:", search_config(config, "ntp server 10.0.0.1"))
```

```
# Exact match — should return False
print("Public SNMP present:", search_config(config, "snmp-server comm

# Regex match — catches both 'transport input telnet' and 'transport
print("Telnet enabled:", search_config(config, "transport input.*telr
```

The expected output should be:

```
NTP server present: True
Public SNMP present: False
Telnet enabled: False
```

A common mistake is when you are working with special characters in regex, like `~!@#$$%^&*()` for example. If your pattern contains a special character and you mean it literally, you have to escape it with a backslash. Another common mistake is case sensitivity. You can get around this by sanitizing the input with the `.lower()` function.

So if:

```
data = "Hello World"
print(data)
```

Your output would be:

```
Hello World
```

If you want to make everything lowercase, you can do this:

```
data = "Hello World"
lowercasedData = data.lower()
print(lowercasedData)
```

Your output would be:

```
hello world
```

This function is important because it is reused in Module 4 with `check_config_contains` and `check_config_not_contains`. The rule handlers will need this exact logic. Solve the search issue once so it doesn't need to be rebuilt for every function.

Module 3 Question

1. Why does `search_config` accept a `match_type` parameter instead of always using regex?
-

Module 4: The Engine

Objectives:

- Build the rule filter (`rule_applies`)
- Write three rule handler functions
- Build a Cisco style config parser
- Use a dispatch table to route rules to their handlers
- Understand why dispatch tables beat long if/elif/else chains

Background

This is the heart of the audit tool. We have devices (Module 1), configs (Module 2), and a way to search those configs (Module 3). We will now build the audit engine that takes a rule from `policy.yaml`, decides whether it applies to a given device, and runs the right check.

A rule looks like this:

```
- id: NTP-001
  description: NTP server must be configured
  type: config_contains
  pattern: "ntp server 10.0.0.100"
  match_type: exact
  severity: high
```

Different rules need different evaluation logic. A `config_contains` rule searches for a pattern. A `config_not_contains` rule searches for the absence of a pattern. An `check_every_int` rule parses interface blocks and checks each one. The engine has to dispatch to the right logic based on the rule's `type` field.

We'll build it in five pieces:

1. `rule_applies` Checks if a rule applies to a device
2. `check_config_contains` Handles must contain rules
3. `check_config_not_contains` Handles must not contain rules
4. `parse_interfaces` and `check_every_int` Interface checks

Part 1: Rule applicability

Some rules apply to every device. Others only apply to specific roles or sites. For example, a "WAN ACL must be present" rule only makes sense for edge routers, not core switches.

Look at this rule, which has an `applies_to` field:

```
- id: WAN-ACL-001
  description: Edge routers must have inbound ACLs
  type: config_contains
  pattern: "ip access-group WAN_IN in"
  severity: critical
  applies_to:
    role: edge
```

This rule should only run on devices where the `role == "edge"`. For devices in any other role, it should be skipped.

Open `starter/audit.py` and find `rule_applies(rule, device)`:

```
def rule_applies(rule, device):
    #Module 4: Determine if a rule should apply against a given device
    #Return True if no applies_to, otherwise check each key/value
    pass
```

The logic is:

- If the rule has no `applies_to` field, it applies to everything and will return `True`
- If the rule has an `applies_to` field, every key/value pair in it must match the device's corresponding fields and will return `True` if all match, `False` if any don't

Step 1: Handle the no `applies_to` case

If `"applies_to"` isn't a key in the rule dict, the rule applies globally. We can check this with the `in` operator:

```
"applies_to" in rule #True if the key exists
"applies_to" not in rule #True if the key doesn't exist
```

Try it yourself! Inside `rule_applies`, return `True` if `applies_to` is not in `rule`

Solution

```
def rule_applies(rule, device):  
    if "applies_to" not in rule:  
        return True
```

Step 2: Check each condition

We need to make sure that each key/value in `applies_to` matches the device.

For example, if the rule says:

```
applies_to:  
    role: edge  
    site: DC
```

then the device must have `role: edge` AND `site: DC`. If either don't match, the rule doesn't apply.

Loop through `rule["applies_to"]`. For each `key, value` pair, compare to `device.get(key)`. We use `.get()` instead of `[]` because if the device doesn't have that field at all, `.get()` returns `None` instead of crashing.

If a comparison fails, return `False`. If a full loop is completed, then all conditions matched, so return `True`.

Try it yourself! Add a `for` loop after your Step 1 code that iterates

`rule["applies_to"].items()`. Inside the loop, return `False` if `device.get(key) != value`. After the loop, return `True`

Solution

```
def rule_applies(rule, device):  
    if "applies_to" not in rule:  
        return True  
  
    for key, value in rule["applies_to"].items():  
        if device.get(key) != value:  
            return False  
  
    return True
```

Part 2: Rule handlers

Now we will write the functions that evaluate each type of rule. Those include `config_contains`, `config_not_contains`, and `every_interface_has`. Each handler takes a rule and a config string, and returns a tuple of `(status, reason)` where status is `"PASS"` or `"FAIL"` and reason is a human readable string.

Why a tuple? Because the audit report needs both pieces. Status drives the pass/fail count and the color in the table we will build later.

Step 3: `check_config_contains`

This handler will `PASS` if the pattern is found in the config, `FAIL` if not..

We already `search_config` from Module 3. The handler just calls `search_config` and translates the boolean result into a `(status, reason)` tuple.

Look at `check_config_contains`:

```
def check_config_contains(rule, config):
    #Module 4: PASS if pattern found in config
    pass
```

To get values from the rule dict:

- `rule["pattern"]` the pattern to search for
- `rule.get("match_type", "exact")` match type, defaulting to "exact" if not specified

Then call `search_config(config, pattern, match_type)` to get a boolean. If `True`, return `PASS` if `False`, return `FAIL`.

Try it yourself! Add the handler. Use clear reason strings like `"Found pattern: <pattern>"` or `"Pattern not found: <pattern>"`.

Solution

```
def check_config_contains(rule, config):
    pattern = rule["pattern"]
    match_type = rule.get("match_type", "exact")
    found = search_config(config, pattern, match_type)
```

```
if found:
    return ("PASS", f"Found pattern: {pattern}")
else:
    return ("FAIL", f"Pattern not found: {pattern}")
```

Step 4: `check_config_not_contains`

Inverse of Step 3. PASSES if the pattern is absent, FAILs if it's present.

Same logic, but the meaning of `found` is flipped where finding the pattern means the rule was violated.

Try it yourself! Add `check_config_not_contains`. Reason strings like `"Forbidden pattern found: <pattern>"` (FAIL) and `"Pattern correctly absent: <pattern>"` (PASS) read well in reports.

Solution

```
def check_config_not_contains(rule, config):
    pattern = rule["pattern"]
    match_type = rule.get("match_type", "exact")
    found = search_config(config, pattern, match_type)

    if found:
        return ("FAIL", f"Forbidden pattern found: {pattern}")
    else:
        return ("PASS", f"Pattern correctly absent: {pattern}")
```

Notice how similar Steps 3 and 4 are. The only difference is which boolean leads to which status. Handlers are predictable.

Part 3: Interface-level checks

The rule `INT-DESC-001` says "every interface must have a description configured." We need to find each interface block in the config, then check each block individually.

A Cisco config interface block looks like this:

```
interface GigabitEthernet0/0
description Uplink to ISP-A
ip address 203.0.113.2 255.255.255.252
```

```
no shutdown
!
```

The interface "starts" at a line beginning with `interface` and "ends" when we hit a non indented line, like `!` Lines belonging to the interface are indented with a leading space or tab.

We need a helper function to parse the config into a dict of `{interface_name: [list_of_lines]}`. Then a rule handler that checks each interface for the needed field.

Step 5: `parse_interfaces`

Find `parse_interfaces(config)` :

```
def parse_interfaces(config):
    #Module 4: Parse Cisco config into a dict of interface blocks
    pass
```

This function:

1. Initialize an empty dict `interfaces` and a tracking variable `current_iface = None`
2. Loop through each line of the config(`config.splitlines()` returns a list of lines)
3. If the line starts with `"interface "`, we've hit a new interface block, so we save its name as the current key and create an empty list for its lines
4. Or if we're inside an interface (such as `current_iface` is not `None`) and the line starts with whitespace, add it to that interface's list
5. If we're inside an interface and the line doesn't start with whitespace, we've left the block reset `current_iface` to `None`

The `.startswith(...)` string method tests if a string begins with a given prefix. `.strip()` removes leading and trailing whitespace.

Try it yourself! Add `parse_interfaces`. This is the most complex function in the lab so don't get discouraged! Utilize the reference/audit.py as needed!

Solution

```
def parse_interfaces(config):
    interfaces = {}
    current_iface = None

    for line in config.splitlines():
```

```

    if line.startswith("interface "):
        current_iface = line.strip()
        interfaces[current_iface] = []
    elif current_iface is not None:
        if line.startswith(" ") or line.startswith("\t"):
            interfaces[current_iface].append(line.strip())
        else:
            current_iface = None

    return interfaces

```

After running this on `router1.txt`, you'd get:

```

{
    "interface Loopback0": ["description Management Loopback", "ip address 10.0.0.1/24"],
    "interface GigabitEthernet0/0": ["description Uplink to ISP-A", "ip address 192.168.1.1/24"],
    ...
}

```

Step 6: `check_every_interface_has`

Now the rule handler that uses `parse_interfaces`.

How it should work:

1. Get the required field name from `rule["field"]` (like `"description"`)
2. Parse the config into interface blocks
3. For each interface, check if any of its lines contain the required field
4. If all interfaces have the field, PASS. If any don't, FAIL with the list of offending interfaces.

Python's built-in `any()` function returns `True` if any element of an iterable is true. With a generator expression, you can check if any line in an interface contains the field:

```

any(field in line for line in iface_lines)

```

This checks `field in line` for each line and returns `True` as soon as any one is true.

Try it yourself! Implement `check_every_int`. Track missing interfaces in a list, then return `PASS` or `FAIL` based on whether the list is empty.

Solution:

```
def check_every_int(rule, config):
    field = rule["field"]
    interfaces = parse_interfaces(config)

    if not interfaces:
        return ("PASS", "No interfaces found in config")

    missing = []
    for iface_name, iface_lines in interfaces.items():
        if not any(field in line for line in iface_lines):
            missing.append(iface_name)

    if missing:
        return ("FAIL", f"Missing '{field}' on: {' '.join(missing)}")
    else:
        return ("PASS", f"All {len(interfaces)} interfaces have '{field}'")
```

The `' '.join(missing)` part takes a list of strings and concatenates them with `,` between each. So `["Gi0/0", "Gi0/1"]` becomes `"Gi0/0, Gi0/1"`.

Part 4: The dispatch table

We have three handler functions. Now we need to wire them up so `evaluate_rule` calls the right one based on the rule's `type` field.

The functional, but messy way to do this would be a long chain of `if / elif`:

```
def evaluate_rule(rule, config):
    if rule["type"] == "config_contains":
        return check_config_contains(rule, config)
    elif rule["type"] == "config_not_contains":
        return check_config_not_contains(rule, config)
    elif rule["type"] == "every_interface_has":
        return check_every_interface_has(rule, config)
    else:
        return ("FAIL", f"Unknown rule type: {rule['type']}")
```

This works. But it has problems like:

- Every new rule type means modifying `evaluate_rule`
- The function grows linearly with the number of rule types
- The dispatch logic is mixed with the function-calling logic

A cleaner approach uses a dispatch table which is a dict mapping rule type strings to handler functions. In Python, functions are first-class values; you can put them in dicts:

```
RULE_HANDLERS = {
    "config_contains": check_config_contains,
    "config_not_contains": check_config_not_contains,
    "every_interface_has": check_every_interface_has,
}
```

Then `evaluate_rule` becomes:

```
def evaluate_rule(rule, config):
    handler = RULE_HANDLERS.get(rule["type"])
    if handler is None:
        return ("FAIL", f"Unknown rule type: {rule['type']}")
    return handler(rule, config)
```

This function uses four lines instead of nine and adding a new rule type means writing a new function and adding one line to the dispatch table. `evaluate_rule` itself never changes.

Step 7: Build the dispatch table

Find the `rule_handler`

```
rule_handler = {
    #Module 4: Map rule type strings to handler functions
}
```

Try it yourself! Add three entries mapping each rule type string to the matching handler function. **Don't include parentheses after the function names.** `check_config_contains` (without parens) is the function itself; `check_config_contains()` (with parens) would call it.

Solution

```
rule_handler = {
    "config_contains": check_config_contains,
    "config_not_contains": check_config_not_contains,
    "every_interface_has": check_every_interface_has,
}
```

Step 8: `evaluate_rule`

Find `evaluate_rule`:

```
def evaluate_rule(rule, config):  
    #Module 4: Look up the right handler and call it  
    pass
```

Use `rule_handler.get(rule["type"])` to grab the handler. The `.get()` method returns `None` if the key isn't found, which lets us handle unknown rule types gracefully.

Try it yourself! Add `evaluate_rule`.

Solution:

```
def evaluate_rule(rule, config):  
    handler = rule_handler.get(rule["type"])  
    if handler is None:  
        return ("FAIL", f"Unknown rule type: {rule['type']}")  
    return handler(rule, config)
```

Testing the engine

Update your test block:

```
if __name__ == "__main__":  
    inventory = load_inventory("starter/inventory.yaml")  
    policy = load_policy("starter/policy.yaml")  
    config = load_config(inventory[0]) # router1's config  
  
    for rule in policy["rules"]:  
        if rule_applies(rule, inventory[0]):  
            status, reason = evaluate_rule(rule, config)  
            print(f"{rule['id']:15} | {status} | {reason}")  
        else:  
            print(f"{rule['id']:15} | N/A | Does not apply")
```

Run the test block. For `router1`, you should see all five rules pass:

```
NTP-001          | PASS | Found pattern: ntp server 10.0.0.100  
TELNET-001       | PASS | Pattern correctly absent: transport input.*telnet
```

SNMP-001	PASS	Pattern correctly absent: snmp-server community
BANNER-001	PASS	Found pattern: banner login
INT-DESC-001	PASS	All 4 interfaces have 'description'

A common mistake is calling a function in the dispatch table. Make sure to not use parantheses in the dispatch table.

Module 4 Questions

1. Why does the dispatch table store function references instead of function calls? What's the difference?
2. The `rule_applies` function uses `device.get(key)` instead of `device[key]`. What error would `device[key]` throw if the device doesn't have that field, and how does `.get()` avoid it?

Module 5: Running the Audit

Objectives:

- Loop over multiple devices and rules
- Gather results in a list of dicts
- Handle errors with try/except so one failure doesn't kill the audit engine
- Use the `continue` keyword to skip iterations

Background

In Module 4 we built the rule engine, where given one rule and one config, it returns a result. `run_audit` loops over every device, loads each config, and evaluates every applicable rule. The output is a list of result dicts that the reporting module (Module 6) will render as a table.

The function has to handle three cases per (device, rule) pair:

1. The rule applies and evaluates to PASS or FAIL and records the result
2. The rule doesn't apply to this device and records N/A
3. The device's config file is missing and records ERROR for every rule on that device

We'll use Python's `try/except` to catch the missing file case, and the `continue` keyword to skip iterations.

Building the result structure

Each result is a dict with five fields:

```
{
    "device": "router1",
    "rule_id": "NTP-001",
    "severity": "high",
    "status": "PASS",
    "reason": "Found pattern: ntp server 10.0.0.100"
}
```

Why are we using a list of dicts and not a list of tuples? Because dicts are self documenting.

`result["status"]` reads more clearly than `result[3]`. When the reporting module renders the table, it accesses fields by name, which makes it much harder to introduce position bugs.

Writing `run_audit()`

Open `starter/audit.py` and find the `run_audit` function:

```
def run_audit(inventory, policy):
    #Module 5: Run every rule against every device, collect results
    pass
```

The function takes the inventory list (from Module 1) and the policy dict (from Module 1). It returns a list of result dicts.

We'll build it in four steps.

Step 1: Set up the outer loop

Start by creating an empty results list, then loop over each device.

Try it yourself! Inside `run_audit`, create an empty list named `results`, then write a `for` loop that iterates over `inventory`. We can leave the body of the loop empty.

Solution

```
def run_audit(inventory, policy):
    results = []

    for device in inventory:
        pass

    return results
```

Step 2: Load the device's config

Inside the loop, load the config using `load_config(device)`. Wrap it in `try/except` so if the config file doesn't exist, we want to record an ERROR for every rule on that device and skip to the next device.

The `try/except` pattern looks like:

```
try:
    risky_operation()
except SpecificError:
    handle_failure()
```

For any missing files, the specific error is `FileNotFoundError`.

When the file is missing, we still want the audit to produce results for that device and just marked as ERROR. Loop through every rule in `policy["rules"]` and add an error result for each, then use `continue` to jump to the next device.

The `continue` keyword skips the rest of the loop body and starts the next iteration. Without `continue` we'd fall through and try to evaluate rules against a config that doesn't exist.

Try it yourself! Wrap `load_config(device)` in `try/except FileNotFoundError`. In the except block, append an ERROR result for each rule, then `continue`.

Solution

```
def run_audit(inventory, policy):
    results = []

    for device in inventory:
        try:
            config = load_config(device)
        except FileNotFoundError:
            for rule in policy["rules"]:
                results.append({
                    "device": device["name"],
                    "rule_id": rule["id"],
                    "severity": rule["severity"],
                    "status": "ERROR",
                    "reason": f"Config file not found: {device['config_path']}"
                })
            continue
```

```
return results
```

`run_audit` is now complete!

Tuple unpacking

The line:

```
status, reason = evaluate_rule(rule, config)
```

is called tuple unpacking. `evaluate_rule` returns a 2-tuple like `("PASS", "Found pattern: ...")`. Instead of writing:

```
result = evaluate_rule(rule, config)
status = result[0]
reason = result[1]
```

we can unpack both values in one line. Python sees the comma on the left side and assigns positionally so the first element to `status`, second to `reason`.

Testing `run_audit`

Your test block should now be:

```
if __name__ == "__main__":
    inventory = load_inventory("starter/inventory.yaml")
    policy = load_policy("starter/policy.yaml")
    results = run_audit(inventory, policy)

    print(f"Total checks: {len(results)}")
    print()
    for r in results:
        print(f"{r['device']:10} | {r['rule_id']:15} | {r['status']:5} |
```

Running that, you should see the following output:

```
Total checks: 15
router1      | NTP-001          | PASS  | Found pattern: ntp server 10.0.0.1
router1      | TELNET-001       | PASS  | Pattern correctly absent: transpor
```

router1		SNMP-001		PASS		Pattern correctly absent: snmp-ser
router1		BANNER-001		PASS		Found pattern: banner login
router1		INT-DESC-001		PASS		All 4 interfaces have 'descriptor
switch1		NTP-001		PASS		Found pattern: ntp server 10.0.0.1
switch1		TELNET-001		FAIL		Forbidden pattern found: transport

To verify if the `FileNotFoundError` handling works, rename `router1.txt` to `router1_off.txt` and run the script. You should see `ERROR` for router1 and run normally for the other devices. Rename back to `router1.txt` after testing.

A common mistake is forgetting `continue`. This can crash the program since Python won't know what to do with an error condition.

Module 5 Questions

1. What does the `continue` keyword do? Why do we need it after the `except` block in Step 2?
 2. Why does `run_audit` return a list of dicts rather than printing results directly?
-

Module 6: Reporting

Objectives:

- Use the `rich` library to render formatted output
- Build a table dynamically from a list of dicts
- Apply colors based on data values
- Compute statistics
- Wire everything together with a `main()` function and entry-point guard

Background

In Module 5 we made a list of result dicts. That list is correct, but raw and printing it as plain text isn't useful for an engineer scanning for failures. We want a real audit report that has a table with columns, color-coded statuses (green for `PASS`, red for `FAIL`), a header showing what policy was applied, and a summary line.

Python's standard `print()` could do the job, but the result would be ugly and manually padding columns and computing widths is tedious. The `rich` library handles all of that automatically. It also supports inline color markup that works in any modern terminal.

How to use `rich`

Three things from `rich` are useful for us with this lab:

- **Console** is the main object you print through. Anything you print with it gets `rich`'s formatting treatment.
- **Table** handles column alignment, borders, and width calculation automatically. You add columns with `add_column` and rows with `add_row`.
- **Inline markup tags** `rich` parses bracket tags in strings, similar to how HTML `[bold]hello[/bold]` renders as bold text. `[green]ok[/green]` renders green. Anything outside the tags renders normally.

An example:

```
from rich.console import Console
from rich.table import Table

console = Console()
console.print("[bold]Audit Report[/bold]")

table = Table()
table.add_column("Name")
table.add_column("Status")
table.add_row("router1", "[green]PASS[/green]")
table.add_row("router2", "[red]FAIL[/red]")
console.print(table)
```

Coloring by status

We want `PASS` rows green, `FAIL` rows red, `N/A` rows dim, and `ERROR` rows yellow.

```
status_styles = {
    "PASS": "green",
    "FAIL": "red",
    "N/A": "dim",
    "ERROR": "yellow",
}
```

When building each row, look up the style for the row's status and wrap the text in markup tags.

Writing `print_report()`

Open `starter/audit.py` and find the `print_report` function:

```
def print_report(results, policy):  
    #Module 6: Render results as a colored table  
    pass
```

Make sure your imports at the top of the file include:

```
from rich.console import Console  
from rich.table import Table
```

We'll build the function in five steps.

Step 1: Print the report header

Start by creating a `Console` and printing a header showing the policy's metadata (name, version, author).

The policy dict from `load_policy` has a `metadata` sub dict with these fields. Access them as `policy["metadata"]["name"]`

Try it yourself! Inside `print_report`, create a `Console` named `console`. Print a bold header like `"Network Compliance Audit Report"`, then print the policy name and version on one line, then the author on another.

Solution

```
def print_report(results, policy):  
    console = Console()  
  
    meta = policy["metadata"]  
    console.print()  
    console.print("[bold]Network Compliance Audit Report[/bold]")  
    console.print(f"Policy: {meta['name']} v{meta['version']}")  
    console.print(f"Author: {meta['author']}")  
    console.print()
```

The empty `console.print()` calls add blank lines for spacing.

Step 2: Build the table structure

Create a `Table` with five columns, `Device`, `Rule ID`, `Severity`, `Status`, and `Reason`.

The `add_column` method accepts options. `style="cyan"` colors the entire column. `no_wrap=True` keeps short fields from line wrapping poorly.

Try it yourself Create a table named `table` with five columns. Make the Device column cyan and prevent wrapping on the first four columns

Solution

```
table = Table(show_lines=False)
table.add_column("Device", style="cyan", no_wrap=True)
table.add_column("Rule ID", no_wrap=True)
table.add_column("Severity", no_wrap=True)
table.add_column("Status", no_wrap=True)
table.add_column("Reason")
```

`show_lines=False` makes a cleaner table without horizontal lines between every row.

Step 3: Add a row per result

Loop through the results list. For each result, look up the style based on the status and add a row.

To look up the style, use `style = status_styles.get(r["status"], "white")`. The second argument to `.get()` is a default so if the status isn't in the dict, fall back to white.

The status cell wraps the status text in color markup: `f"[{style}]{r['status']}\n[/{style}]"`. For example this `"[green]PASS[/green]"` makes green.

For severity, uppercase it for visual emphasis: `r["severity"].upper()`.

Try it yourself! Loop through `results` and call `table.add_row` for each one. Build the Status cell with color markup based on `status_styles`.

Solution

```
for r in results:
    style = status_styles.get(r["status"], "white")
    table.add_row(
        r["device"],
        r["rule_id"],
        r["severity"].upper(),
        f"[{style}]{r['status']}\n[/{style}]",
        r["reason"],
    )
```

```
console.print(table)
```

Step 4: Add the summary line

Under the table, print a one line summary with counts of each status. We compute the counts using generator expressions like:

```
passed = sum(1 for r in results if r["status"] == "PASS")
```

This generates a `1` for each result with status `"PASS"` and sums them. This is equal to `len([r for r in results if r["status"] == "PASS"])` but doesn't build an intermediate list.

Try it yourself! Compute the four counts and print a summary like `"Summary: 15 checks - 8 pass, 7 fail, 0 skipped, 0 errored"` with color tags around the numbers.

Solution:

```
total = len(results)
passed = sum(1 for r in results if r["status"] == "PASS")
failed = sum(1 for r in results if r["status"] == "FAIL")
skipped = sum(1 for r in results if r["status"] == "N/A")
errored = sum(1 for r in results if r["status"] == "ERROR")

console.print()
console.print(f"[bold]Summary:[/bold] {total} checks - "
              f"[green]{passed} pass[/green], "
              f"[red]{failed} fail[/red], "
              f"[dim]{skipped} skipped[/dim], "
              f"[yellow]{errored} errored[/yellow]")
console.print()
```

Notice the multi line f string. Python automatically concatenates adjacent string literals on the same line so these four `f"..."` strings become one big string at runtime.

Step 5: The `main()` function and entry point

The `main()` function puts together the whole program, with the functions load inventory, load policy, run audit, and print report.

Find the `main` function:

```
def main():
    inventory = load_inventory("starter/inventory.yaml")
    policy = load_policy("starter/policy.yaml")
    results = run_audit(inventory, policy)
    print_report(results, policy)

if __name__ == "__main__":
    main()
```

Final test

Run the complete script:

```
python starter/audit.py
```

You should see a colorized table.

Module 5 Questions

1. The `status_styles` dict lives at module level, not inside `print_report`. Why?
 2. Why use `rich` instead of just printing with regular `print()` calls?
-

Next steps

You've built a working tool. Some next steps to grow the program:

- **Live device support.** Replace the file-based `load_config` with a Netmiko version that pulls configs over SSH.
 - **More rule types.** Add `interface_count_at_least`, `ips_in_range`. Each one is a new function and a new entry in the dispatch table.
 - **HTML or JSON output.** Write a new `render_report_html(results, policy)` function that produces a styled HTML file instead of terminal output.
 - **Rule parameterization.** Currently rules are static. Imagine parameters like `{ntp_server}` that get substituted from a per-site configuration before evaluation.
 - **CI integration.** Run the audit on every config commit using GitHub Actions or GitLab CI. Auto-fail merges that introduce non-compliant configs.
-

